```
!pip install scikit-learn tensorflow transformers torch datasets seaborn matplotlib
```

```
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.po
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tens
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transfo
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.19.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorfl
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->te
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->te
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tenso
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tens
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tenso
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingfac
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0-
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hu
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->ke
```

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
from datasets import load_dataset

dataset = load_dataset("imdb")

# Shuffle dataset
train_dataset_shuffled = dataset['train'].shuffle(seed=42)
test_dataset_shuffled = dataset['test'].shuffle(seed=42)

# Use more data
train_texts = train_dataset_shuffled['text'][:20000]
train_labels = train_dataset_shuffled['label'][:20000]

test_texts = test_dataset_shuffled['text'][:5000]
test_labels = test_dataset_shuffled['label'][:5000]
```

```
import time
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Better TF-IDF configuration
vectorizer = TfidfVectorizer(
    max_features=20000,      # larger vocabulary
    ngram_range=(1,2),       # unigram + bigram
    stop_words='english',    # remove common words
    min_df=3,                # remove rare noise words
    max_df=0.9               # ignore very frequent words
)

X_train = vectorizer.fit_transform(train_texts)
X_test = vectorizer.transform(test_texts)

print("Unique labels:", np.unique(train_labels))

start = time.time()

lr_model = LogisticRegression(
    max_iter=800,
    solver='liblinear'
)

lr_model.fit(X_train, train_labels)

lr_train_time = time.time() - start

lr_preds = lr_model.predict(X_test)

lr_acc = accuracy_score(test_labels, lr_preds)

print("Logistic Regression Accuracy:", lr_acc)
print("Training time:", lr_train_time)
```

```
Unique labels: [0 1]
Logistic Regression Accuracy: 0.8748
Training time: 0.7848377227783203
```

```
import time
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from sklearn.metrics import accuracy_score

max_words = 10000
max_len = 300   # increased sequence length

tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(train_texts)

X_train_seq = tokenizer.texts_to_sequences(train_texts)
X_test_seq = tokenizer.texts_to_sequences(test_texts)

X_train_pad = pad_sequences(X_train_seq, maxlen=max_len)
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len)

start = time.time()

lstm_model = Sequential([
    Embedding(max_words, 128, input_length=max_len),
    LSTM(128),          # increased LSTM units
    Dropout(0.5),       # helps generalization
    Dense(1, activation='sigmoid')
])

lstm_model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

lstm_model.fit(
    X_train_pad,
    np.array(train_labels),
    epochs=3,           # increased epochs
    batch_size=64,
    validation_split=0.2
```

```
    validation_split=0.2
)

lstm_train_time = time.time() - start

lstm_preds = (lstm_model.predict(X_test_pad) > 0.5).astype(int)

lstm_acc = accuracy_score(test_labels, lstm_preds)

print("LSTM Accuracy:", lstm_acc)
print("Training time:", lstm_train_time)
```

```
Epoch 1/3
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is depre
  warnings.warn(
250/250 ━━━━━━━━━━━━━━━━━━━━ 9s 21ms/step - accuracy: 0.6270 - loss: 0.6296 - val_accuracy: 0.8455 - val_loss: 0.3633
Epoch 2/3
250/250 ━━━━━━━━━━━━━━━━━━━━ 5s 20ms/step - accuracy: 0.8901 - loss: 0.2792 - val_accuracy: 0.8680 - val_loss: 0.3397
Epoch 3/3
250/250 ━━━━━━━━━━━━━━━━━━━━ 5s 19ms/step - accuracy: 0.9098 - loss: 0.2425 - val_accuracy: 0.8457 - val_loss: 0.5068
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step
LSTM Accuracy: 0.829
Training time: 18.40060830116272
```

```
import time
import numpy as np
import torch
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import Trainer, TrainingArguments
from datasets import Dataset
from sklearn.metrics import accuracy_score

model_name = "bert-base-uncased"

tokenizer = BertTokenizer.from_pretrained(model_name)

train_dataset = Dataset.from_dict({
    "text": train_texts,
    "label": train_labels
})

test_dataset = Dataset.from_dict({
    "text": test_texts,
    "label": test_labels
})

def tokenize(example):
    return tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=128   # smaller length = faster training
    )

train_dataset = train_dataset.map(tokenize, batched=True)
test_dataset = test_dataset.map(tokenize, batched=True)

train_dataset.set_format(type="torch", columns=["input_ids","attention_mask","label"])
test_dataset.set_format(type="torch", columns=["input_ids","attention_mask","label"])

model = BertForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,                     # more epochs improves accuracy
    per_device_train_batch_size=32,         # larger batch = faster
    per_device_eval_batch_size=32,
    learning_rate=2e-5,
    eval_strategy="epoch",
    logging_steps=50
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset
)

start = time.time()
```

```
trainer.train()

bert_train_time = time.time() - start

preds = trainer.predict(test_dataset)

bert_preds = np.argmax(preds.predictions, axis=1)

bert_acc = accuracy_score(test_labels, bert_preds)

print("BERT Accuracy:", bert_acc)
print("Training time:", bert_train_time)
```

Map: 100% 20000/20000 [00:18<00:00, 1168.52 examples/s]

Map: 100% 5000/5000 [00:05<00:00, 855.34 examples/s]

Loading weights: 100% 199/199 [00:00<00:00, 662.76it/s, Materializing param=bert.pooler.dense.weight]

```
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                          | Status     |
---------------------------------------------+------------+-
cls.predictions.transform.dense.bias         | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
cls.predictions.bias                         | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
classifier.bias                              | MISSING    |
classifier.weight                            | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
```

[1875/1875 27:16, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	0.300445	0.280604
2	0.210387	0.282706
3	0.117197	0.362181

Writing model shards: 100% 1/1 [00:32<00:00, 32.67s/it]

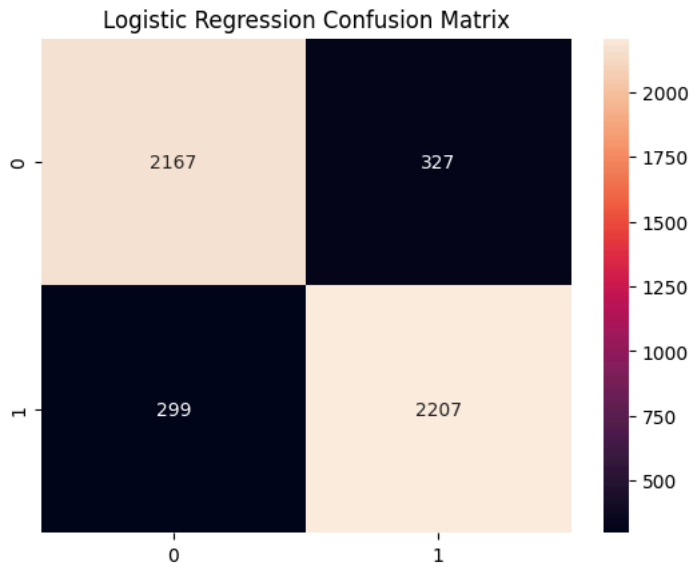
Writing model shards: 100% 1/1 [00:27<00:00, 27.24s/it]

Writing model shards: 100% 1/1 [00:41<00:00, 41.61s/it]

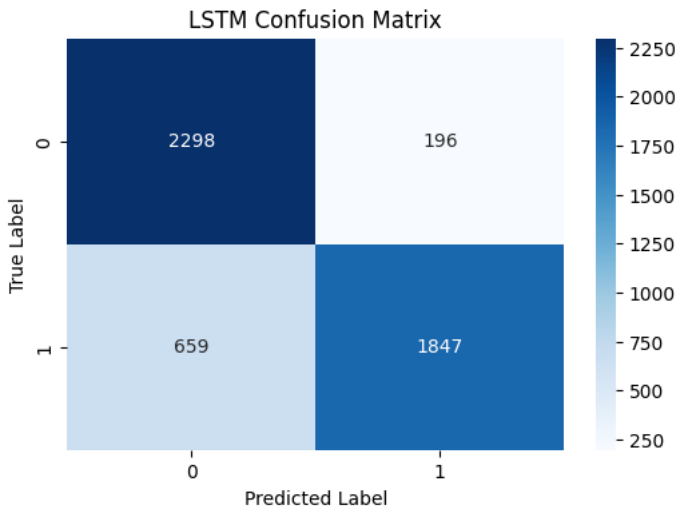
Writing model shards: 100% 1/1 [00:22<00:00, 22.10s/it]

```
BERT Accuracy: 0.8908
Training time: 1638.1523129940033
```

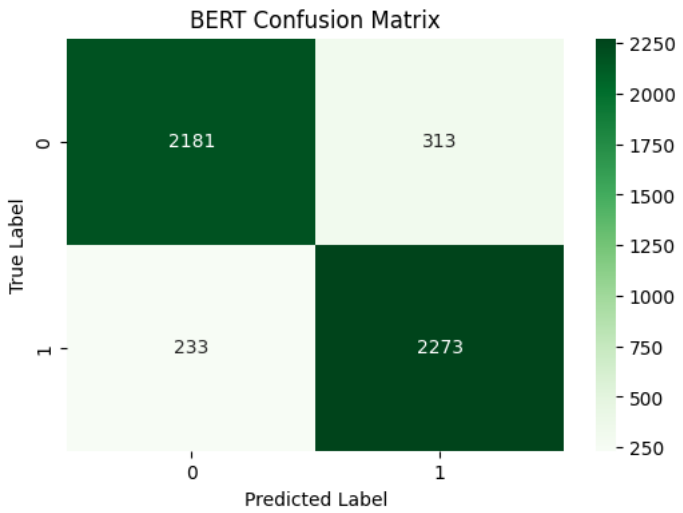
```
cm = confusion_matrix(test_labels, lr_preds)

sns.heatmap(cm, annot=True, fmt="d")
plt.title("Logistic Regression Confusion Matrix")
plt.show()
```



```
lstm_preds = lstm_preds.flatten()

cm_lstm = confusion_matrix(test_labels, lstm_preds)

plt.figure(figsize=(6,4))
sns.heatmap(cm_lstm, annot=True, fmt="d", cmap="Blues")

plt.title("LSTM Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.show()
```




```
cm_bert = confusion_matrix(test_labels, bert_preds)

plt.figure(figsize=(6,4))
sns.heatmap(cm_bert, annot=True, fmt="d", cmap="Greens")

plt.title("BERT Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.show()
```




```
print("Logistic Regression")
print(classification_report(test_labels, lr_preds))

print("LSTM")
print(classification_report(test_labels, lstm_preds))

print("BERT")
print(classification_report(test_labels, bert_preds))
```

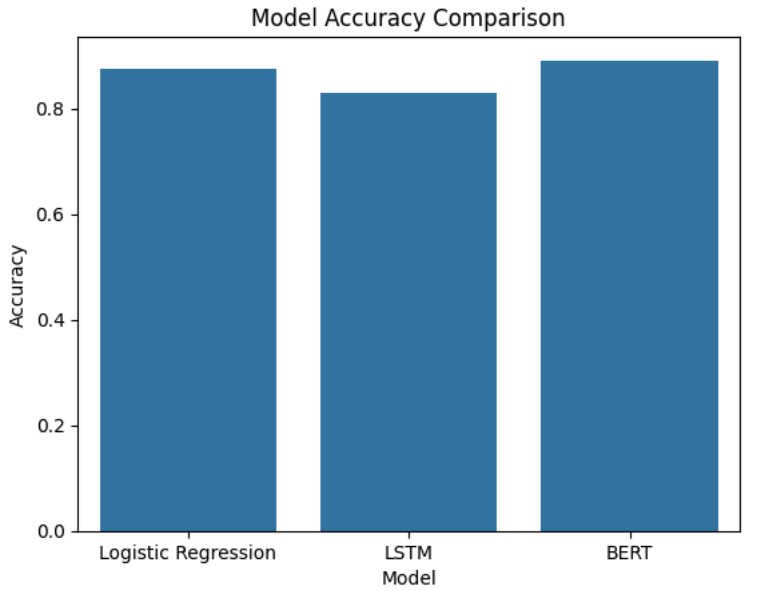
```
Logistic Regression
              precision    recall  f1-score   support

           0       0.88      0.87      0.87      2494
```

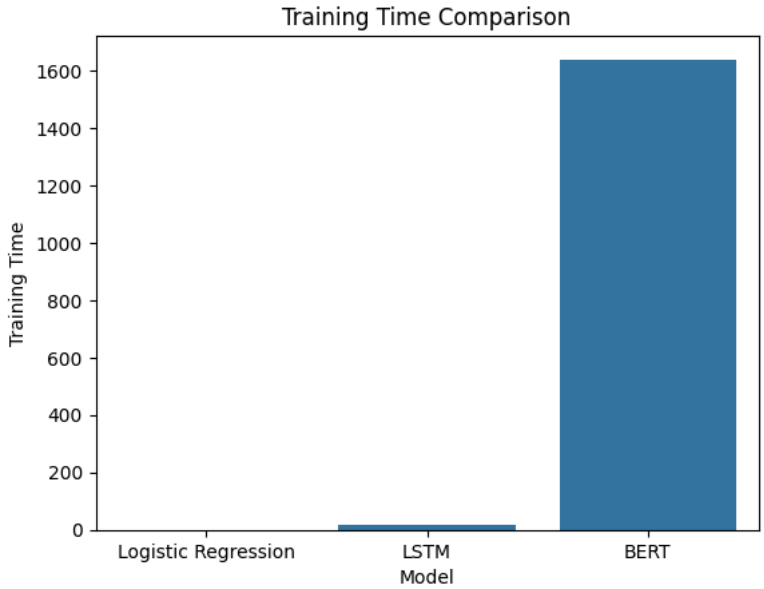
```
           1       0.87      0.88      0.88      2506

    accuracy                           0.87      5000
   macro avg       0.87      0.87      0.87      5000
weighted avg       0.87      0.87      0.87      5000

LSTM
              precision    recall  f1-score   support

           0       0.78      0.92      0.84      2494
           1       0.90      0.74      0.81      2506

    accuracy                           0.83      5000
   macro avg       0.84      0.83      0.83      5000
weighted avg       0.84      0.83      0.83      5000

BERT
              precision    recall  f1-score   support

           0       0.90      0.87      0.89      2494
           1       0.88      0.91      0.89      2506

    accuracy                           0.89      5000
   macro avg       0.89      0.89      0.89      5000
weighted avg       0.89      0.89      0.89      5000
```

```
results = pd.DataFrame({
    "Model": ["Logistic Regression","LSTM","BERT"],
    "Accuracy": [lr_acc, lstm_acc, bert_acc],
    "Training Time": [lr_train_time, lstm_train_time, bert_train_time]
})

print(results)
```

```
                 Model  Accuracy  Training Time
0  Logistic Regression    0.8748       0.784838
1                 LSTM    0.8290      18.400608
2                 BERT    0.8908    1638.152313
```

```
sns.barplot(x="Model", y="Accuracy", data=results)
plt.title("Model Accuracy Comparison")
plt.show()

sns.barplot(x="Model", y="Training Time", data=results)
plt.title("Training Time Comparison")
plt.show()
```






```
import pandas as pd

lr_error_df = pd.DataFrame({
    "review": test_texts,
    "true_label": test_labels,
    "predicted_label": lr_preds
})

# False Positives
lr_fp = lr_error_df[
    (lr_error_df.true_label == 0) &
    (lr_error_df.predicted_label == 1)
]

# False Negatives
lr_fn = lr_error_df[
    (lr_error_df.true_label == 1) &
    (lr_error_df.predicted_label == 0)
]

print("Logistic Regression Errors")
print("False Positives:", len(lr_fp))
print("False Negatives:", len(lr_fn))
```

```
Logistic Regression Errors
False Positives: 327
False Negatives: 299
```

```
print("Example Logistic Regression Errors:\n")

for i in range(3):
    print("Review:", lr_fn.iloc[i]["review"][:400])
```

```
    print("True Label:", lr_fn.iloc[i]["true_label"])
    print("Predicted:", lr_fn.iloc[i]["predicted_label"])
    print("-"*80)
```

```
Example Logistic Regression Errors:

Review: This is a really sad, and touching movie! It deals with the subject of child abuse. It's really sad, but mostly a tr
True Label: 1
Predicted: 0
--------------------------------------------------------------------------------
Review: Late night on BBC1, was on my way to bed but curiosity piqued at a contemporary-set Irish film so I stayed to watch
True Label: 1
Predicted: 0
--------------------------------------------------------------------------------
Review: At first I didn't think that the performance by Lauren Ambrose was anything but flaky, but as her character develope
True Label: 1
Predicted: 0
--------------------------------------------------------------------------------
```

```
lstm_error_df = pd.DataFrame({
    "review": test_texts,
    "true_label": test_labels,
    "predicted_label": lstm_preds.flatten()
})

lstm_fp = lstm_error_df[
    (lstm_error_df.true_label == 0) &
    (lstm_error_df.predicted_label == 1)
]

lstm_fn = lstm_error_df[
    (lstm_error_df.true_label == 1) &
    (lstm_error_df.predicted_label == 0)
]

print("LSTM Errors")
print("False Positives:", len(lstm_fp))
print("False Negatives:", len(lstm_fn))
```

```
LSTM Errors
False Positives: 196
False Negatives: 659
```

```
print("Example LSTM Errors:\n")

for i in range(3):
    print("Review:", lstm_fn.iloc[i]["review"][:400])
    print("True Label:", lstm_fn.iloc[i]["true_label"])
    print("Predicted:", lstm_fn.iloc[i]["predicted_label"])
    print("-"*80)
```

```
Example LSTM Errors:

Review: This is the latest entry in the long series of films with the French agent, O.S.S. 117 (the French answer to James B
True Label: 1
Predicted: 0
--------------------------------------------------------------------------------
Review: After a very long time Marathi cinema has come with some good movie.This movie is one of the best Marathi movies eve
True Label: 1
Predicted: 0
--------------------------------------------------------------------------------
Review: This is a really sad, and touching movie! It deals with the subject of child abuse. It's really sad, but mostly a tr
True Label: 1
Predicted: 0
--------------------------------------------------------------------------------
```

```
bert_error_df = pd.DataFrame({
    "review": test_texts,
    "true_label": test_labels,
    "predicted_label": bert_preds
})

bert_fp = bert_error_df[
    (bert_error_df.true_label == 0) &
    (bert_error_df.predicted_label == 1)
]

bert_fn = bert_error_df[
    (bert_error_df.true_label == 1) &
    (bert_error_df.predicted_label == 0)
]
```

```
print("BERT Errors")
print("False Positives:", len(bert_fp))
print("False Negatives:", len(bert_fn))
```

```
BERT Errors
False Positives: 313
False Negatives: 233
```

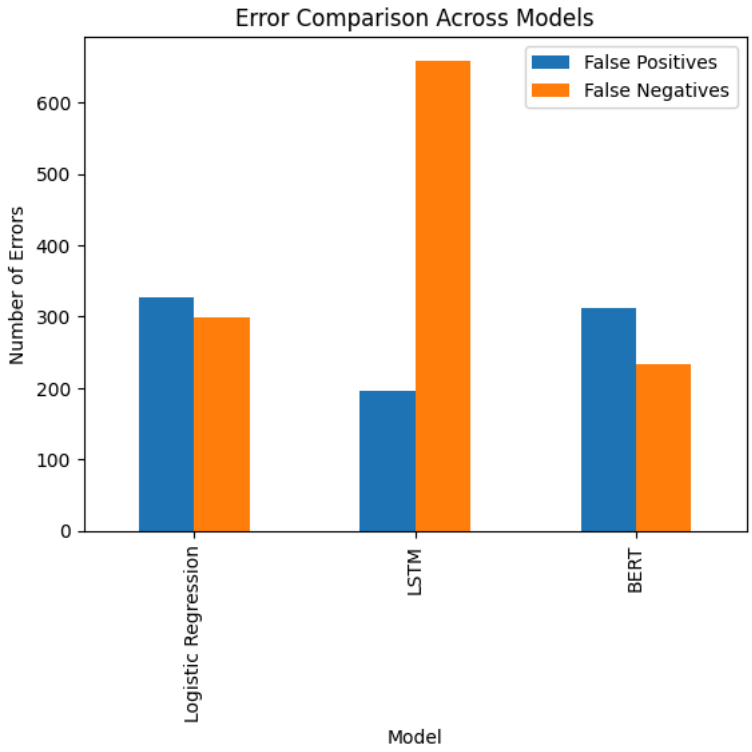
```
print("Example BERT Errors:\n")

for i in range(3):
    print("Review:", bert_fn.iloc[i]["review"][:400])
    print("True Label:", bert_fn.iloc[i]["true_label"])
    print("Predicted:", bert_fn.iloc[i]["predicted_label"])
    print("-"*80)
```

```
Example BERT Errors:

Review: Late night on BBC1, was on my way to bed but curiosity piqued at a contemporary-set Irish film so I stayed to watch
True Label: 1
Predicted: 0
--------------------------------------------------------------------------------
Review: I'd like to point out these excellent points in favor of this movie:<br /><br />#1 Angelina Jolie sex scene <br /><b
True Label: 1
Predicted: 0
--------------------------------------------------------------------------------
Review: Sex, drugs, racism and of course you ABC's. What more could you want in a kid's show!<br /><br />----------
True Label: 1
Predicted: 0
--------------------------------------------------------------------------------
```

```
error_summary = pd.DataFrame({
    "Model": ["Logistic Regression", "LSTM", "BERT"],
    "False Positives": [len(lr_fp), len(lstm_fp), len(bert_fp)],
    "False Negatives": [len(lr_fn), len(lstm_fn), len(bert_fn)]
})

print(error_summary)
```

```
                 Model  False Positives  False Negatives
0  Logistic Regression              327              299
1                 LSTM              196              659
2                 BERT              313              233
```

```
import matplotlib.pyplot as plt

error_summary.set_index("Model").plot(kind="bar")

plt.title("Error Comparison Across Models")
plt.ylabel("Number of Errors")
plt.xlabel("Model")

plt.show()
```




```
import random

def add_noise(text):
    words = text.split()
    random.shuffle(words)
    return " ".join(words)

noisy_test = [add_noise(t) for t in test_texts]

X_test_noise = vectorizer.transform(noisy_test)

noise_preds = lr_model.predict(X_test_noise)

print("Logistic Regression Accuracy on Noisy Data:",
      accuracy_score(test_labels, noise_preds))
```

```
Logistic Regression Accuracy on Noisy Data: 0.8702
```

```
from tensorflow.keras.preprocessing.text import Tokenizer

max_words = 10000

keras_tokenizer = Tokenizer(num_words=max_words)

# Fit tokenizer on training text
keras_tokenizer.fit_on_texts(train_texts)
```

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.metrics import accuracy_score

# Convert noisy text to sequences
noisy_seq = keras_tokenizer.texts_to_sequences(noisy_test)

# Pad sequences
X_test_noise_lstm = pad_sequences(noisy_seq, maxlen=max_len)

# Predict
lstm_noise_preds = (lstm_model.predict(X_test_noise_lstm) > 0.5).astype(int)

# Flatten predictions
lstm_noise_preds = lstm_noise_preds.flatten()

print("LSTM Accuracy on Noisy Data:",
      accuracy_score(test_labels, lstm_noise_preds))
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step
LSTM Accuracy on Noisy Data: 0.8252
```

```
from transformers import BertTokenizer

bert_tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
from sklearn.metrics import accuracy_score
import torch
import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

batch_size = 16
bert_noise_preds = []

for i in range(0, len(noisy_test), batch_size):

    batch_texts = noisy_test[i:i+batch_size]

    encoded = bert_tokenizer(
        batch_texts,
        padding=True,
        truncation=True,
        max_length=256,
        return_tensors="pt"
    )

    encoded = {k: v.to(device) for k, v in encoded.items()}

    with torch.no_grad():
        outputs = model(**encoded)

    preds = torch.argmax(outputs.logits, dim=1)

    bert_noise_preds.extend(preds.cpu().numpy())

print("BERT Accuracy on Noisy Data:",
      accuracy_score(test_labels, bert_noise_preds))
```

```
BERT Accuracy on Noisy Data: 0.8136
```

```
import pandas as pd
from sklearn.metrics import accuracy_score

noise_results = pd.DataFrame({
    "Model": ["Logistic Regression", "LSTM", "BERT"],
    "Noisy Accuracy": [
        accuracy_score(test_labels, noise_preds),
        accuracy_score(test_labels, lstm_noise_preds),
        accuracy_score(test_labels, bert_noise_preds)
    ]
})
```